

Assignment Submission: Task 2

Program: BSCS-8B

Project: AI-Generated Retro Snake Game

Date: June 10, 2026

Deployed Game Link: [Try Game!](#)

Development Report: AI Generated Retro Snake Game

Overview

This document outlines the development process of a Retro Snake game built using an AI coding assistant. The project was created specifically to explore AI-assisted development workflows, iterative prompt engineering, and rapid prototyping.

Prompt Iterations Used During Development

The game was developed through a series of progressive prompts, starting with core mechanics and moving toward polish and platform compatibility.

1. Iteration 1: Foundation & Aesthetics

- **Prompt Focus:** Core game loop (WASD/arrows, score, growth, collisions), obstacles, skin selection, 3 distinct maps (Lava, Greenery, Neon), menu system, and a specific "retro/arcade" visual style.
- **Result:** The AI generated a functional React app using HTML5 <canvas>, but introduced a bug where the snake immediately collided with obstacles on the Lava and Neon maps upon starting.

2. Iteration 2: Bug Fixing & Feature Expansion

- **Prompt Focus:** Fixing the instant "Game Over" bug on the Neon and Lava maps. Adding difficulty levels (Easy, Normal, Hard) to control the initial speed.
- **Result:** The AI adjusted the coordinate arrays for obstacles in the config file so they no longer overlapped with the starting position of the snake. Difficulty states were successfully wired into the loop's speed setTimeout.

3. Iteration 3: Audio Engineering

- **Prompt Focus:** Adding sound effects for eating and game over, retro background music, a global mute toggle, and displaying the current difficulty on the canvas.
- **Result:** The AI implemented a custom AudioEngine using the native Web Audio API (avoiding bulky external assets) and attached initialization to user interaction to bypass modern browser autoplay restrictions.

4. Iteration 4: Mobile & Touch Support

- **Prompt Focus:** Making the game mobile-responsive and allowing touch/swipe controls (up, down, left, right) for mobile users.
- **Result:** Touch event listeners (touchstart, touchend) were added to calculate swipe delta (dx, dy), alongside CSS adjustments (overscroll-behavior: none) to prevent the browser window from scrolling while playing.

5. Iteration 5: Documentation

- **Prompt Focus:** Generating the README file with an ASCII art banner and this final development report.

What Worked Well

- **React + HTML5 Canvas Combination:** React handled the overarching state (Menus, Score, Difficulty, Skins) perfectly, while the native <canvas> API allowed for highly efficient, pixel-perfect rendering of the game board.
- **Component Architecture:** The AI instinctively separated the data (config.ts), the game rendering (GameCanvas.tsx), the audio (audio.ts), and the UI state (App.tsx). This made it incredibly easy to inject new features (like Audio and Difficulty) later without breaking the whole application.
- **Aesthetics Customization:** The Tailwind CSS configuration combined with custom CSS animations successfully captured the requested 80s CRT monitor visual style (scanlines and flickers) accurately on the first attempt.

What Did Not Work

- **Initial Hardcoded Map Layouts:** In the first iteration, the AI hardcoded the obstacle coordinates. It randomly placed obstacles directly on the snake's default spawning coordinates (10, 10). Consequently, starting the Neon or Lava map triggered an instant collision and "Game Over."
- **Mobile Viewport Scrolling:** When touch controls were initially considered, standard touch interactions on websites caused the page to refresh or scroll, which disrupted the gameplay experience until specific CSS overscroll constraints were implemented.

How Prompts Were Improved Over Time

- **From Broad to Specific:** The initial prompt was a massive conceptual dump covering game rules, themes, deployment info, and colors. While beneficial for initial setup, subsequent prompts became highly targeted (e.g., "Neon grid map not working," "add sound effects").
- **Providing Contextual Bug Reports:** Instead of stating "the game is broken," the prompts specifically identified *which* maps were failing ("only mountain greenery able to start game"). This allowed the AI to immediately pinpoint the issue (obstacle coordinate overlap) rather than rewriting the entire game logic.
- **Chaining Feature Requests:** Rather than asking for everything at once, features were requested step-by-step. Once the core game worked, difficulty was added. Once difficulty worked, audio was added. Once audio worked, mobile controls were requested.

Challenges Faced During the Process

1. **Browser Audio Policies:** Browsers strictly block audio from playing automatically before the user interacts with the page. The audio system had to be carefully refactored to initialize the AudioContext only after the user pressed a key, clicked a button, or touched the screen.
2. **Touch Swipe Calculation:** Translating a mobile swipe into a discrete grid direction (Up, Down, Left, Right) required calculating the absolute difference between starting and ending coordinates (dx and dy) and ensuring the game logic prioritized the axis with the larger movement.
3. **Responsive Scaling:** Keeping the HTML5 canvas proportional on smaller mobile screens while maintaining pixel-perfect retro rendering required careful CSS container sizing and preventing mobile double-tap-to-zoom default behaviors.